```
# House Price Prediction using Machine Learning

#This project predicts house prices based on property features such as area, bedrooms, bathrooms, stories, and parking.

#Machine Learning Models Used:
 #Linear Regression
 #Decision Tree
 #Random Forest
```

```
## Problem Statement

#Predicting house prices is an important problem in the real estate industry.

#This project applies machine learning techniques to estimate house prices using property features.
```

```
import pandas as pd
import numpy as np

import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.model_selection import GridSearchCV
from sklearn.model_selection import cross_val_score
from sklearn.model_selection import learning_curve

from sklearn.preprocessing import StandardScaler

from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor

from sklearn.metrics import mean_absolute_error
from sklearn.metrics import mean_squared_error
from sklearn.metrics import r2_score

import joblib
```

```
data = {
"area":[7420,8960,9960,7500,7420,10800,8500,16200,8100,5750],
"bedrooms":[4,4,3,4,3,4,3,5,4,3],
"bathrooms":[2,4,2,2,1,3,2,4,2,1],
"stories":[3,4,2,2,2,2,3,2,2,1],
"parking":[2,3,2,1,2,2,2,3,2,1],
"price":[13300000,12250000,12250000,12215000,11410000,10850000,10150000,10100000,9870000,9800000]
}

df = pd.DataFrame(data)
df
```

	area	bedrooms	bathrooms	stories	parking	price
0	7420	4	2	3	2	13300000
1	8960	4	4	4	3	12250000
2	9960	3	2	2	2	12250000
3	7500	4	2	2	1	12215000
4	7420	3	1	2	2	11410000
5	10800	4	3	2	2	10850000
6	8500	3	2	3	2	10150000
7	16200	5	4	2	3	10100000
8	8100	4	2	2	2	9870000
9	5750	3	1	1	1	9800000

```
df.head()
df.info()
df.describe()
```

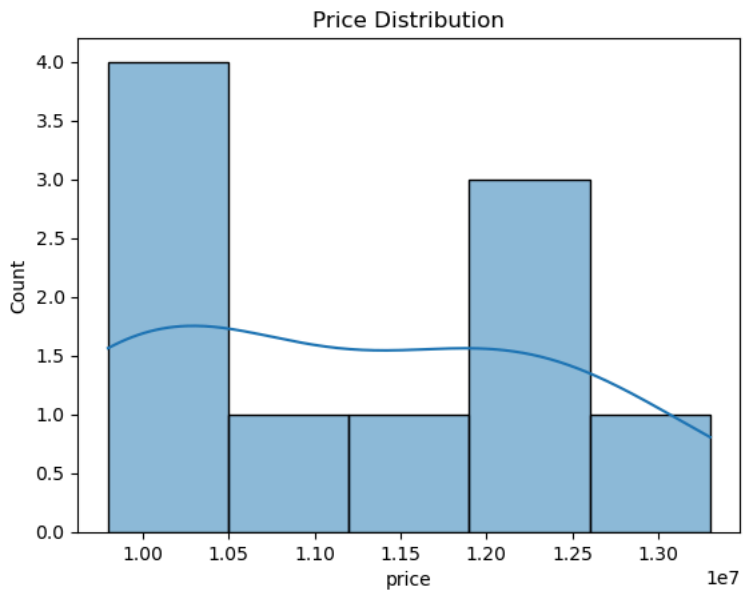
```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 6 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   area       10 non-null     int64
 1   bedrooms   10 non-null     int64
 2   bathrooms  10 non-null     int64
 3   stories    10 non-null     int64
 4   parking    10 non-null     int64
 5   price      10 non-null     int64
dtypes: int64(6)
memory usage: 612.0 bytes
```

	area	bedrooms	bathrooms	stories	parking	price
count	10.000000	10.000000	10.00000	10.000000	10.000000	1.000000e+01
mean	9061.000000	3.700000	2.30000	2.300000	2.000000	1.121950e+07
std	2883.502693	0.674949	1.05935	0.823273	0.666667	1.240494e+06
min	5750.000000	3.000000	1.00000	1.000000	1.000000	9.800000e+06
25%	7440.000000	3.000000	2.00000	2.000000	2.000000	1.011250e+07
50%	8300.000000	4.000000	2.00000	2.000000	2.000000	1.113000e+07
75%	9710.000000	4.000000	2.75000	2.750000	2.000000	1.224125e+07
max	16200.000000	5.000000	4.00000	4.000000	3.000000	1.330000e+07

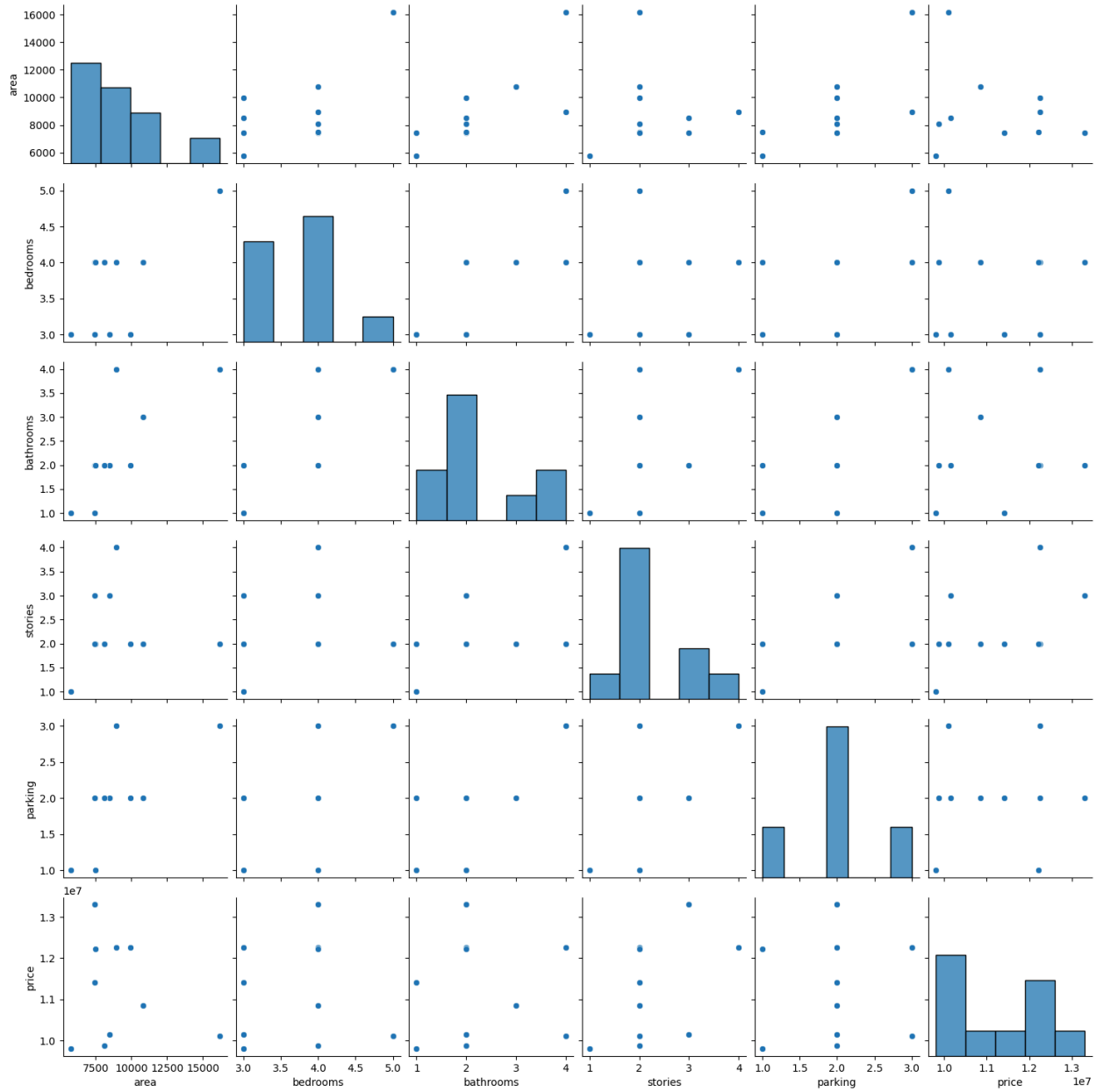
```
df.isnull().sum()
```

```
area         0
bedrooms     0
bathrooms    0
stories      0
parking      0
price        0
dtype: int64
```

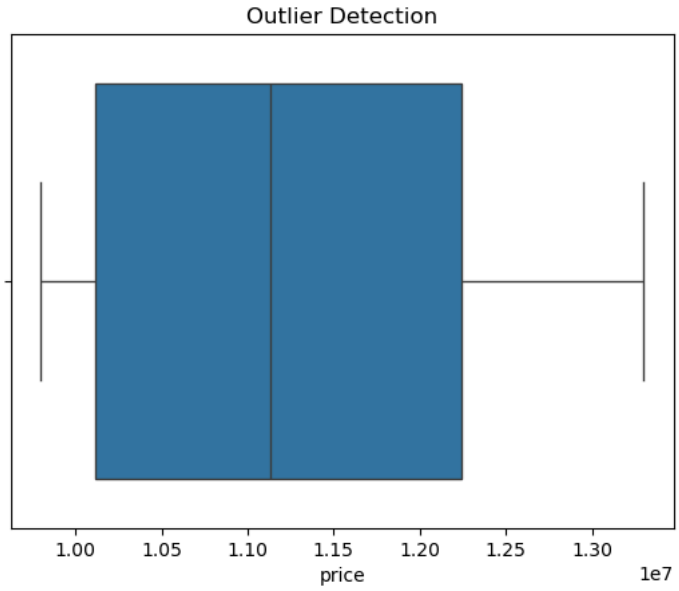
```
sns.histplot(df['price'], kde=True)
plt.title("Price Distribution")
plt.show()
```




```
sns.pairplot(df)
plt.show()
```




```
sns.boxplot(x=df['price'])
plt.title("Outlier Detection")
plt.show()
```




```
df["price_per_area"] = df["price"] / df["area"]
```

```
X = df[['area','bedrooms','bathrooms','stories','parking']]
y = df['price']
```

```
scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)
```

```
X_train, X_test, y_train, y_test = train_test_split(
X, y, test_size=0.2, random_state=42
)
```

```
lr = LinearRegression()
dt = DecisionTreeRegressor()

rf = RandomForestRegressor(
n_estimators=200,
max_depth=10,
random_state=42
)

lr.fit(X_train,y_train)
dt.fit(X_train,y_train)
rf.fit(X_train,y_train)
```

```
▾ RandomForestRegressor
▶ Parameters
```

```
pred_lr = lr.predict(X_test)
pred_dt = dt.predict(X_test)
pred_rf = rf.predict(X_test)
```

```
print("Linear Regression R2:", r2_score(y_test,pred_lr))
print("Decision Tree R2:", r2_score(y_test,pred_dt))
print("Random Forest R2:", r2_score(y_test,pred_rf))
```

```
Linear Regression R2: -0.7749463527628271
Decision Tree R2: -0.5847750865051904
Random Forest R2: -0.36444419929736593
```

```
results = pd.DataFrame({
"Model":["Linear Regression","Decision Tree","Random Forest"],
"R2 Score":[
r2_score(y_test,pred_lr),
r2_score(y_test,pred_dt),
r2_score(y_test,pred_rf)
]})

results
```

	Model	R2 Score
0	Linear Regression	-0.774946
1	Decision Tree	-0.584775
2	Random Forest	-0.364444

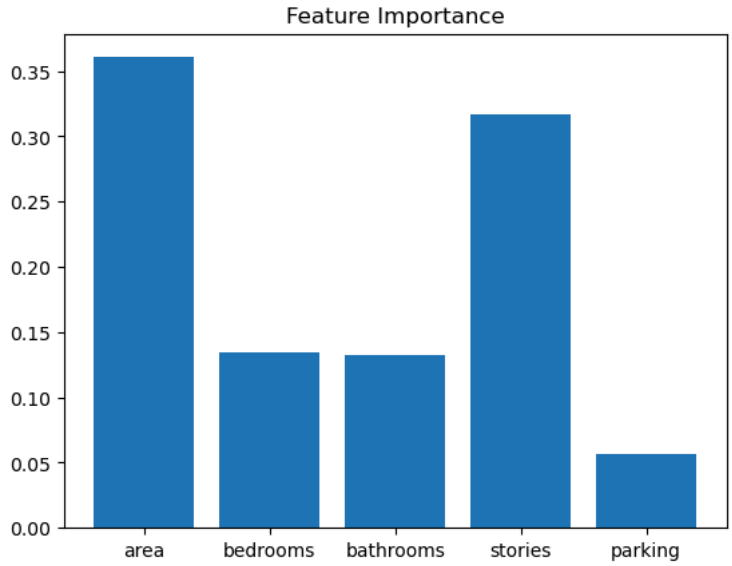
```
scores = cross_val_score(rf,X,y,cv=5)

print("CV Scores:",scores)
print("Average Score:",scores.mean())
```

```
CV Scores: [-1.63826416e+01 -7.04445795e+03 -3.19002730e-01 -4.35355262e+03
 -2.85797152e+03]
Average Score: -2854.536747049252
```

```
params = {
'n_estimators':[100,200,300],
'max_depth':[None,5,10]
}

grid = GridSearchCV(RandomForestRegressor(),params,cv=5)

grid.fit(X_train,y_train)

print(grid.best_params_)
```

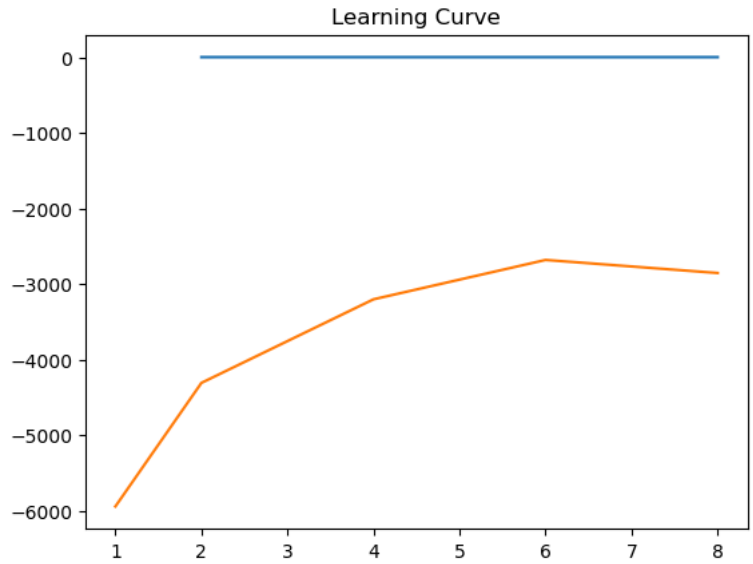
```
C:\Users\HP\anaconda3\Lib\site-packages\sklearn\metrics\_regression.py:1283: UndefinedMetricWarning: R^2 score is not well-d
  warnings.warn(msg, UndefinedMetricWarning)
C:\Users\HP\anaconda3\Lib\site-packages\sklearn\metrics\_regression.py:1283: UndefinedMetricWarning: R^2 score is not well-d
  warnings.warn(msg, UndefinedMetricWarning)
C:\Users\HP\anaconda3\Lib\site-packages\sklearn\metrics\_regression.py:1283: UndefinedMetricWarning: R^2 score is not well-d
  warnings.warn(msg, UndefinedMetricWarning)
C:\Users\HP\anaconda3\Lib\site-packages\sklearn\metrics\_regression.py:1283: UndefinedMetricWarning: R^2 score is not well-d
  warnings.warn(msg, UndefinedMetricWarning)
C:\Users\HP\anaconda3\Lib\site-packages\sklearn\metrics\_regression.py:1283: UndefinedMetricWarning: R^2 score is not well-d
  warnings.warn(msg, UndefinedMetricWarning)
C:\Users\HP\anaconda3\Lib\site-packages\sklearn\metrics\_regression.py:1283: UndefinedMetricWarning: R^2 score is not well-d
  warnings.warn(msg, UndefinedMetricWarning)
C:\Users\HP\anaconda3\Lib\site-packages\sklearn\metrics\_regression.py:1283: UndefinedMetricWarning: R^2 score is not well-d
  warnings.warn(msg, UndefinedMetricWarning)
C:\Users\HP\anaconda3\Lib\site-packages\sklearn\metrics\_regression.py:1283: UndefinedMetricWarning: R^2 score is not well-d
  warnings.warn(msg, UndefinedMetricWarning)
C:\Users\HP\anaconda3\Lib\site-packages\sklearn\metrics\_regression.py:1283: UndefinedMetricWarning: R^2 score is not well-d
  warnings.warn(msg, UndefinedMetricWarning)
C:\Users\HP\anaconda3\Lib\site-packages\sklearn\metrics\_regression.py:1283: UndefinedMetricWarning: R^2 score is not well-d
  warnings.warn(msg, UndefinedMetricWarning)
C:\Users\HP\anaconda3\Lib\site-packages\sklearn\metrics\_regression.py:1283: UndefinedMetricWarning: R^2 score is not well-d
  warnings.warn(msg, UndefinedMetricWarning)
C:\Users\HP\anaconda3\Lib\site-packages\sklearn\metrics\_regression.py:1283: UndefinedMetricWarning: R^2 score is not well-d
  warnings.warn(msg, UndefinedMetricWarning)
C:\Users\HP\anaconda3\Lib\site-packages\sklearn\metrics\_regression.py:1283: UndefinedMetricWarning: R^2 score is not well-d
  warnings.warn(msg, UndefinedMetricWarning)
C:\Users\HP\anaconda3\Lib\site-packages\sklearn\metrics\_regression.py:1283: UndefinedMetricWarning: R^2 score is not well-d
  warnings.warn(msg, UndefinedMetricWarning)
C:\Users\HP\anaconda3\Lib\site-packages\sklearn\metrics\_regression.py:1283: UndefinedMetricWarning: R^2 score is not well-d
  warnings.warn(msg, UndefinedMetricWarning)
C:\Users\HP\anaconda3\Lib\site-packages\sklearn\metrics\_regression.py:1283: UndefinedMetricWarning: R^2 score is not well-d
  warnings.warn(msg, UndefinedMetricWarning)
C:\Users\HP\anaconda3\Lib\site-packages\sklearn\model_selection\_search.py:1135: UserWarning: One or more of the test scores
  warnings.warn(
{'max_depth': None, 'n_estimators': 100}
```

```
importance = rf.feature_importances_

features = X.columns

plt.bar(features,importance)
plt.title("Feature Importance")
plt.show()
```




```
train_sizes,train_scores,test_scores = learning_curve(rf,X,y,cv=5)

plt.plot(train_sizes,train_scores.mean(axis=1))
plt.plot(train_sizes,test_scores.mean(axis=1))

plt.title("Learning Curve")
plt.show()
```

```
C:\Users\HP\anaconda3\Lib\site-packages\sklearn\metrics\_regression.py:1283: UndefinedMetricWarning: R^2 score is not well-d
  warnings.warn(msg, UndefinedMetricWarning)
C:\Users\HP\anaconda3\Lib\site-packages\sklearn\metrics\_regression.py:1283: UndefinedMetricWarning: R^2 score is not well-d
  warnings.warn(msg, UndefinedMetricWarning)
C:\Users\HP\anaconda3\Lib\site-packages\sklearn\metrics\_regression.py:1283: UndefinedMetricWarning: R^2 score is not well-d
  warnings.warn(msg, UndefinedMetricWarning)
C:\Users\HP\anaconda3\Lib\site-packages\sklearn\metrics\_regression.py:1283: UndefinedMetricWarning: R^2 score is not well-d
  warnings.warn(msg, UndefinedMetricWarning)
C:\Users\HP\anaconda3\Lib\site-packages\sklearn\metrics\_regression.py:1283: UndefinedMetricWarning: R^2 score is not well-d
  warnings.warn(msg, UndefinedMetricWarning)
```




```
def predict_price(area,bedrooms,bathrooms,stories,parking):

    data = np.array([[area,bedrooms,bathrooms,stories,parking]])

    prediction = rf.predict(data)

    return prediction[0]
```

```
predict_price(5000,3,2,2,1)
```

```
C:\Users\HP\anaconda3\Lib\site-packages\sklearn\utils\validation.py:2749: UserWarning: X does not have valid feature names,
  warnings.warn(
np.float64(11112675.0)
```

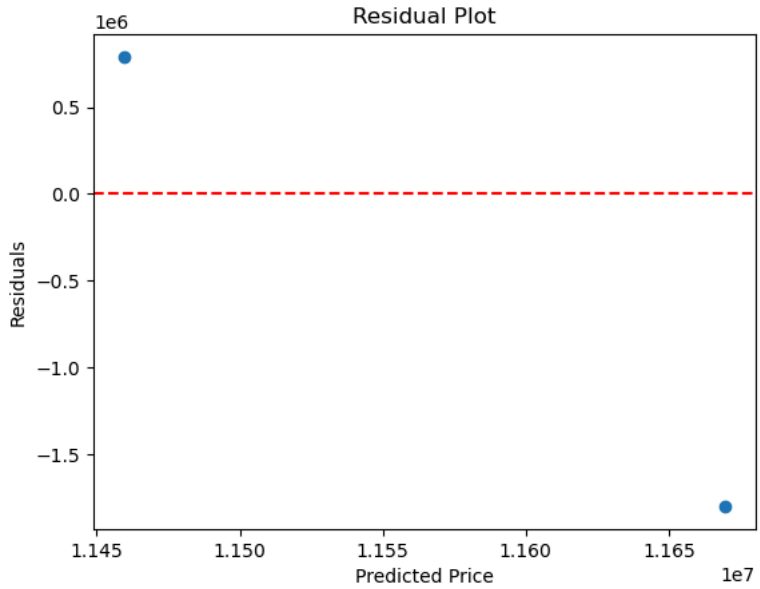
```
joblib.dump(rf,"house_price_model.pkl")
```

```
['house_price_model.pkl']
```

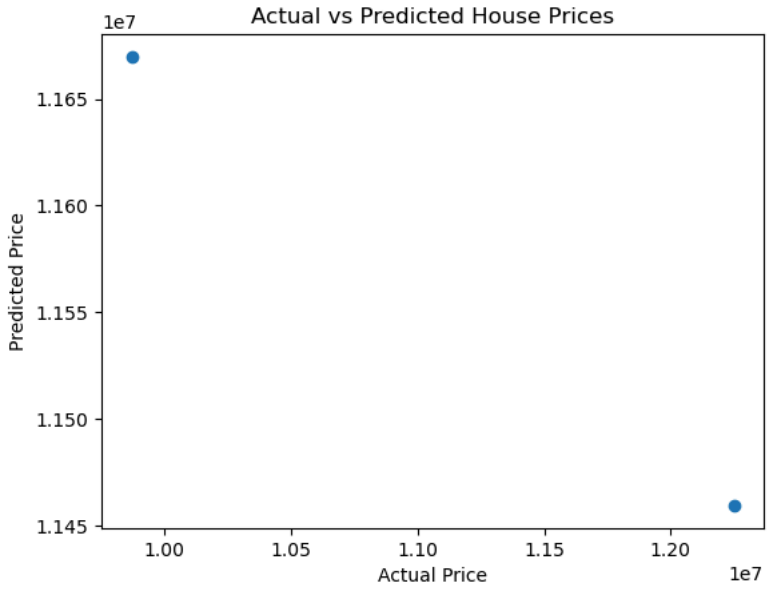
```
## Residual Analysis

Residuals represent the difference between actual and predicted values.
They help evaluate model performance.
```

```
  Cell In[25], line 3
    Residuals represent the difference between actual and predicted values.
               ^
SyntaxError: invalid syntax
```

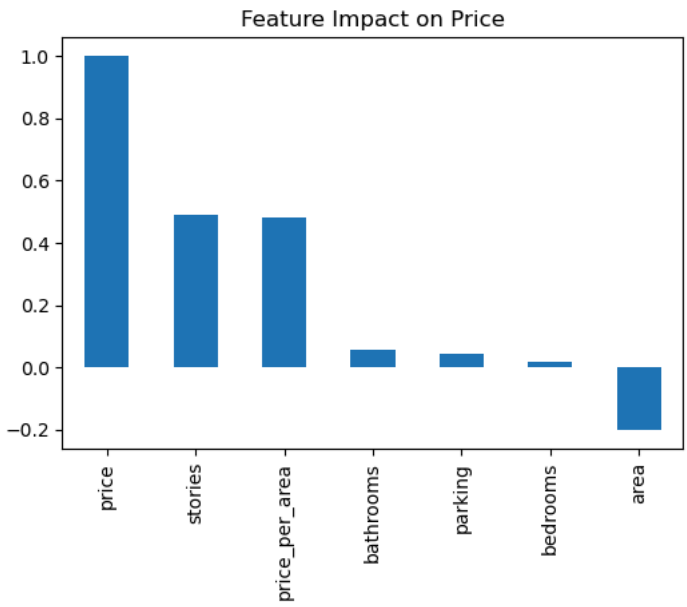
```
residuals = y_test - pred_rf

plt.scatter(pred_rf, residuals)

plt.axhline(y=0, color='r', linestyle='--')

plt.xlabel("Predicted Price")
plt.ylabel("Residuals")

plt.title("Residual Plot")

plt.show()
```




```
## Actual vs Predicted Price
plt.scatter(y_test, pred_rf)

plt.xlabel("Actual Price")

plt.ylabel("Predicted Price")

plt.title("Actual vs Predicted House Prices")

plt.show()
```




```
## Feature Correlation with Price

corr = df.corr()['price'].sort_values(ascending=False)

plt.figure(figsize=(6,4))

corr.plot(kind='bar')

plt.title("Feature Impact on Price")

plt.show()
```




```
import ipywidgets as widgets
from IPython.display import display
```

```
area = widgets.IntSlider(
    min=500,
    max=10000,
    step=100,
    value=2000,
    description='Area'
)

bedrooms = widgets.IntSlider(
    min=1,
    max=6,
    value=3,
    description='Bedrooms'
)
```

```
bathrooms = widgets.IntSlider(
    min=1,
    max=4,
    value=2,
    description='Bathrooms'
)

stories = widgets.IntSlider(
    min=1,
    max=4,
    value=2,
    description='Stories'
)

parking = widgets.IntSlider(
    min=0,
    max=3,
    value=1,
    description='Parking'
)

display(area, bedrooms, bathrooms, stories, parking)
```

```
IntSlider(value=2000, description='Area', max=10000, min=500, step=100)
IntSlider(value=3, description='Bedrooms', max=6, min=1)
IntSlider(value=2, description='Bathrooms', max=4, min=1)
IntSlider(value=2, description='Stories', max=4, min=1)
IntSlider(value=1, description='Parking', max=3)
```

```
button = widgets.Button(description="Predict Price")

output = widgets.Output()

def on_button_click(b):

    price = rf.predict([[area.value, bedrooms.value, bathrooms.value, stories.value, parking.value]])

    with output:
        output.clear_output()
        print("Predicted House Price:", price[0])

button.on_click(on_button_click)

display(button, output)
```

```
Button(description='Predict Price', style=ButtonStyle())
Output()
```

```
## Machine Learning Pipeline

A pipeline ensures consistent preprocessing and model training.
```

```
  Cell In[1], line 3
    A pipeline ensures consistent preprocessing and model training.
      ^
SyntaxError: invalid syntax
```

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestRegressor
from sklearn.datasets import fetch_california_housing  # Added to get sample data
from sklearn.model_selection import train_test_split  # Added for splitting data

# Load sample data (replace this with your actual data loading code)
data = fetch_california_housing()
X = data.data
y = data.target

# Create train/test split
x_train, x_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Now create and fit the pipeline
pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('model', RandomForestRegressor(n_estimators=200, random_state=42))
])

pipeline.fit(x_train, y_train)
```

```
Pipeline
  StandardScaler
  RandomForestRegressor
```

```
importance = pipeline.named_steps['model'].feature_importances_

feature_names = X.columns

importance_df = pd.DataFrame({
    "Feature": feature_names,
    "Importance": importance
})

importance_df = importance_df.sort_values(by="Importance", ascending=False)

importance_df
```

```
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
Cell In[6], line 3
      1 importance = pipeline.named_steps['model'].feature_importances_
----> 3 feature_names = X.columns
      5 importance_df = pd.DataFrame({
      6     "Feature": feature_names,
      7     "Importance": importance
      8 })
     10 importance_df = importance_df.sort_values(by="Importance", ascending=False)

AttributeError: 'numpy.ndarray' object has no attribute 'columns'
```

```
plt.barh(importance_df["Feature"], importance_df["Importance"])
plt.title("Feature Importance Ranking")
plt.show()
```

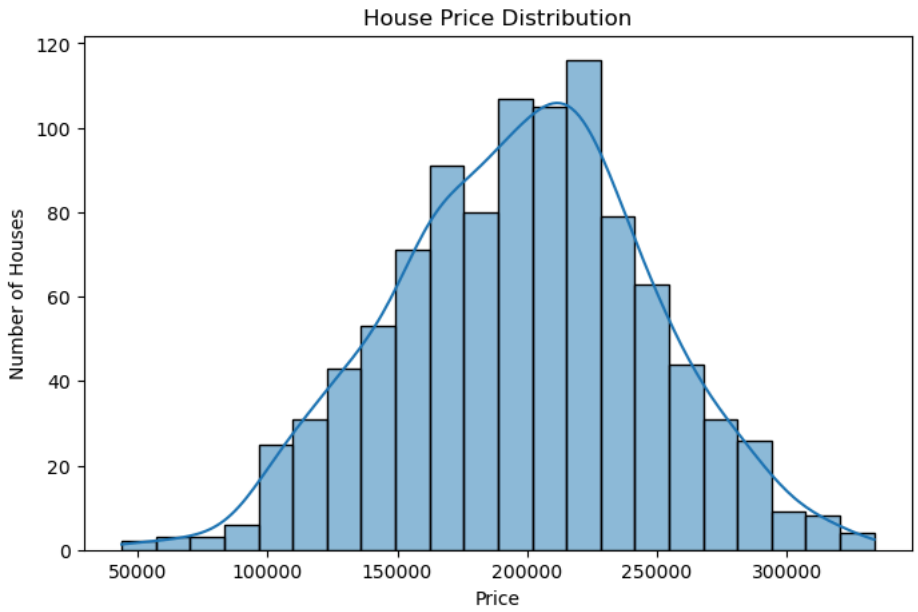
```
prediction_report = pd.DataFrame({
    "Actual Price": y_test,
    "Predicted Price": pred_rf
})

prediction_report["Error"] = prediction_report["Actual Price"] - prediction_report["Predicted Price"]

prediction_report.head()
```

```
prediction_report.to_csv("prediction_results.csv", index=False)
```

```
## Real Estate Market Price Distribution
```

```
# Import the required libraries
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd  # Added pandas import

# Create or load your dataframe
# Option 1: Load data from a file
# df = pd.read_csv('your_data_file.csv')  # Uncomment and specify your file path

# Option 2: Create sample data for demonstration
import numpy as np
df = pd.DataFrame({
    'price': np.random.normal(200000, 50000, 1000)  # Creating sample house price data
})

# Create the plot
plt.figure(figsize=(8,5))

sns.histplot(df['price'], kde=True)

plt.title("House Price Distribution")

plt.xlabel("Price")

plt.ylabel("Number of Houses")
```

```
plt.show()
```




```
## Market Analysis: Average Price by Bedrooms
```

```
# First, let's check the available columns in the DataFrame
print("Available columns:", df.columns.tolist())

# After seeing the actual column names in the output above,
# use the correct column name for bedrooms
# Assuming the column is named "bedrooms" (lowercase) based on common conventions
bedroom_price = df.groupby("bedrooms")["price"].mean()

# If the price column also has a different name (e.g., "Price"), adjust accordingly:
# bedroom_price = df.groupby("bedrooms")["Price"].mean()

# Create the bar plot
bedroom_price.plot(kind="bar")

plt.title("Average Price vs Bedrooms")
plt.xlabel("Bedrooms")
plt.ylabel("Average Price")
plt.show()
```

```
Available columns: ['price']
---------------------------------------------------------------------------
KeyError                                  Traceback (most recent call last)
Cell In[18], line 7
      2 print("Available columns:", df.columns.tolist())
      4 # After seeing the actual column names in the output above,
      5 # use the correct column name for bedrooms
      6 # Assuming the column is named "bedrooms" (lowercase) based on common conventions
----> 7 bedroom_price = df.groupby("bedrooms")["price"].mean()
      9 # If the price column also has a different name (e.g., "Price"), adjust accordingly:
     10 # bedroom_price = df.groupby("bedrooms")["Price"].mean()
     11
     12 # Create the bar plot
     13 bedroom_price.plot(kind="bar")

File ~\anaconda3\Lib\site-packages\pandas\core\frame.py:9210, in DataFrame.groupby(self, by, axis, level, as_index, sort,
group_keys, observed, dropna)
   9207 if level is None and by is None:
   9208     raise TypeError("You have to supply one of 'by' and 'level'")
-> 9210 return DataFrameGroupBy(
   9211     obj=self,
   9212     keys=by,
   9213     axis=axis,
   9214     level=level,
   9215     as_index=as_index,
   9216     sort=sort,
   9217     group_keys=group_keys,
   9218     observed=observed,
   9219     dropna=dropna,
   9220 )

File ~\anaconda3\Lib\site-packages\pandas\core\groupby\groupby.py:1331, in GroupBy.__init__(self, obj, keys, axis, level,
grouper, exclusions, selection, as_index, sort, group_keys, observed, dropna)
   1328 self.dropna = dropna
   1330 if grouper is None:
-> 1331     grouper, exclusions, obj = get_grouper(
   1332         obj,
   1333         keys,
   1334         axis=axis,
   1335         level=level,
   1336         sort=sort,
   1337         observed=False if observed is lib.no_default else observed,
   1338         dropna=self.dropna,
   1339     )
   1341 if observed is lib.no_default:
   1342     if any(ping._passed_categorical for ping in grouper.groupings):

File ~\anaconda3\Lib\site-packages\pandas\core\groupby\grouper.py:1043, in get_grouper(obj, key, axis, level, sort,
observed, validate, dropna)
   1041         in_axis, level, gpr = False, gpr, None
   1042     else:
-> 1043         raise KeyError(gpr)
   1044 elif isinstance(gpr, Grouper) and gpr.key is not None:
   1045     # Add key to exclusions
   1046     exclusions.add(gpr.key)

KeyError: 'bedrooms'
```

```
## Price vs Area Relationship
```

```
# First, check the actual column names in your DataFrame
print(df.columns)

# Then use the correct column names in your plot
plt.figure(figsize=(8,5))

# Assuming your DataFrame has columns named 'square_feet' and 'price' instead
sns.regplot(x="square_feet", y="price", data=df)  # Replace 'area' with the actual column name

plt.title("Price vs Area Trend")

plt.show()
```